An Empirical Analysis on Desynchronization Between Sequence Messages and Timestamps in SPY Market By Order Data

Aaron Gao
University of Waterloo

April 12, 2026

Abstract

This paper investigates the desynchronization between sequence identifiers and network timestamps within SPY Market-By-Order (MBO) data feeds. Our analysis demonstrates that Limit Order Book (LOB) reconstruction relying exclusively on gateway-assigned arrival times leads to erroneous state assumptions and false signals in historical backtesting environments. To resolve this, we propose a maximum cumulative timestamp methodology that guarantees sequential integrity, removing the impact of system jitter and facilitating highly accurate, cross-exchange order book consolidation.

1 Introduction

In high-frequency trading (HFT), the accurate reconstruction of the Limit Order Book (LOB) is a foundational requirement. The validity of order-flow-driven strategies and historical backtesting relies on the adherence to the order in which actions that affect the LOB (add, cancel, modify) are processed. Across the 14 U.S. equity exchanges, direct Market-By-Order (MBO) data feeds typically provide two timeline markers: a gateway-stamped arrival time (`ts_event`) and a matching engine sequence identifier (`sequence`).

While the sequence ID (`sequence`) represents the deterministic execution order within a specific matching engine, it operates in an isolated namespace independent to each exchange. Conversely, the `ts_event` provides a universal chronological timeline but is inaccurate due to system jitter introduced during the handoff between the matching engine and multi-threaded outbound gateways.

This dichotomy presents a challenge when attempting to reconstruct a Central Limit Order Book (CLOB) across all 14 independent exchanges. Cross-exchange consolidation inherently requires merging data via the universal timeline (`ts_event`); however, relying strictly on gateway timestamps often violates the true sequential execution order (`sequence`) within individual exchanges due to system jitter. In this paper, we propose a robust methodology to resolve this structural conflict, guaranteeing strict intra-exchange sequential integrity while maintaining chronological merge compatibility across the broader national market system.

2 Data and Sample Selection

For this empirical analysis, we utilize nanosecond-resolution Market-By-Order (MBO) data for the SPDR S&P 500 ETF Trust (SPY), sourced via Databento. Databento offers collocated historical data feeds from all 14 U.S. equity exchanges.

The primary dataset is a consolidated Parquet file (`spy_mbo.parquet`) containing the tick-level lifecycle of all SPY MBO events for the trading day of March 23, 2026 (2026-03-23).

To ensure computational feasibility for local processing, the analysis focuses on the 30-minute interval from 10:00:00 AM to 10:30:00 AM ET on 2026-03-23.

We utilize DuckDB for efficient out-of-core querying and memory management of the underlying Parquet file. The specific extraction logic is detailed below:

```
query = f"""
SELECT * FROM 'spy_mbo/processed/spy_mbo.parquet'
WHERE ts_event >= '2026-03-23 10:00:00'
  AND ts_event <= '2026-03-23 10:30:00'
"""
market_by_order_df = duckdb.sql(query).df()
```

Listing 1: SQL query to get the 30 minute time sample

The query yields a large sample of 9,752,974 discrete message events. Table 1 outlines the primary columns from the pandas dataframe utilized in this study. Specifically, the gateway arrival timestamps (`ts_event`), matching engine sequence identifiers (`sequence`), and exchange IDs (`publisher_id`) are the essential identifiers used in our research.

ts_event	publisher_id	action	side	price	size	order_id	flags	sequence	symbol
2026-03-23 10:00:00.000344-04:00	43	A	B	660.19	18	284773571068696	128	23939758	SPY
2026-03-23 10:00:00.000353-04:00	43	A	A	660.23	16	284773571068705	128	23939759	SPY
2026-03-23 10:00:00.000560-04:00	5	A	A	660.23	80	3191404863773174854	128	21228992	SPY
2026-03-23 10:00:00.000572-04:00	15	A	A	660.23	80	860838092	128	105650931	SPY
2026-03-23 10:00:00.000624-04:00	2	T	A	660.20	100	0	0	305195009	SPY

Table 1: First 5 rows of the extracted sample

3 Empirical Findings: Desynchronized Order Lifecycles

To empirically demonstrate this desynchronization, we isolate a single order that experienced system jitter during its lifecycle (`order_id = 861805602`):

```
query = f"""
SELECT *
FROM 'spy_mbo/processed/spy_mbo.parquet'
WHERE order_id = 861805602
"""
order_861805602_lifecycle = duckdb.sql(query).df()
order_861805602_lifecycle
```

Listing 2: SQL query to extract the order lifecycle with `order_id` = 861805602

ts_event	publisher_id	action	side	price	size	order_id	flags	sequence	symbol
2026-03-23 10:00:01.896666-04:00	15	C	B	660.14	67	861805602	0	105764171	SPY
2026-03-23 10:00:01.896666-04:00	15	F	B	660.14	67	861805602	0	105764171	SPY
2026-03-23 10:00:01.896666-04:00	15	F	B	660.14	13	861805602	0	105764172	SPY
2026-03-23 10:00:01.896666-04:00	15	C	B	660.14	13	861805602	128	105764172	SPY
2026-03-23 10:00:01.896701-04:00	15	A	B	660.14	80	861805602	0	105764146	SPY

Table 2: Complete table of lifecycle events for `order_id` = 861805602

Before going further in the analysis, it is necessary to clarify the structure of the message block. The four rows depicted represent only two distinct state changes to the Limit Order Book (LOB). When a market order executes against a resting limit order, the exchange matching engine simultaneously broadcasts two discrete messages: an informational Fill message (`action=F`) detailing the trade execution, and a state-altering Cancel message (`action=C`) instructing the LOB to decrement the resting size. Because these derive from the exact same

execution event, they share an identical `sequence` identifier. For the purpose of LOB reconstruction, only the Cancel message is processed to update the book state. Furthermore, because our data loader that generated `spy_mbo.parquet` ordered the dataset by the network arrival timestamp (`ts_event`), messages sharing an identical nanosecond timestamp such as these simultaneous Fill and Cancel pairs are ordered arbitrarily within the dataframe.

As illustrated in Table 2, we isolate the lifecycle of a Bid limit order for 80 shares of SPY at $660.14 on the MEMX exchange (`publisher_id = 15`). If one were to reconstruct this order's timeline relying on the network arrival time (`ts_event`), the chronological sequence would unfold as follows:

- **Timestamp 1 (`10:00:01.896666`):**
  The resting limit order is executed across two discrete partial fills—an initial tranche of 67 shares and a secondary tranche of 13 shares—triggering two corresponding state decrements within the Limit Order Book.

- **Timestamp 2 (`10:00:01.896701`):**
  The matching engine processes the `Add` (`A`) message, instantiating the order and contributing 80 shares of resting depth.

Adhering to this network timeline creates a direct causality violation: the liquidity is formally established 35 nanoseconds *after* its total volume was already absorbed. Because a resting order cannot physically be executed prior to its instantiation, any state reconstruction ordered by network timestamps is flawed. By anchoring the reconstruction to the matching engine's deterministic `sequence` identifier, causality is restored, and the true execution timeline unfolds as follows:

- **Sequence `105764146` (Order Instantiation):**
  The matching engine processes the `Add` event, officially initializing the order's lifecycle and appending 80 shares of resting liquidity to the Limit Order Book.

- **Sequence `105764171` (Primary Execution):**
  An aggressive order matches against the resting liquidity, absorbing an initial tranche of 67 shares. The engine triggers a corresponding state decrement, leaving a residual resting volume of 13 shares.

- **Sequence `105764172` (Secondary Execution):**
  A subsequent match absorbs the remaining 13-share tranche, fully depleting the order's size and clearing it from the active book.

4 Solution: Cumulative Maximum Timestamp over a Window of Sorted Sequence Id Grouped by Exchange Id

To correctly reconstruct the limit order book over historical data, we use the cumulative maximum timestamp. We group the MBO data by `publisher_id` which separates all the data into its individual exchanges. Then we sort the data in the individual exchanges by `sequence`. We must first group the data by its individual exchanges because the `sequence` is independent to every exchange and does not have a global ordering. For the list of data within each exchange sorted by `sequence`, we take the cumulative maximum of `ts_event` to ensure that events with a larger `sequence` will have a larger timestamp. This is necessary because we need to merge the timelines of all 14 exchanges together in order to generate the central limit order book.

5 Crossed Markets and False Liquidity in Limit Order Books

This desynchronization creates crossed markets and liquidity in historic backtesting that never existed during the real time period. During state reconstruction, when the `action=C` (Cancel) message arrives first, the simulation engine skips these events because `order_id = 861805602` does not exist in the active book. When the `action=A` (Add) message arrives second, the 80 shares are instantiated. However, because the order's corresponding cancel messages were already consumed and discarded, this volume becomes permanently stranded. These 80 shares persist indefinitely within the reconstructed Limit Order Book, regardless of how much time has elapsed or where prices have moved.

To empirically demonstrate the effect of desynchronization on both the limit order books of independent exchanges and the consolidated central limit order book, we present a comparative analysis of two state reconstruction methodologies. Both cases simulate a 30-minute historical reconstruction window from 10:00:00 AM to 10:30:00 AM ET.

Case 1: Reconstruction based on `ts_event` (Uncorrected Jitter). In the Jupyter Notebook `backtest_contains_jitter.ipynb`, we reconstruct the limit order book according to the order of `ts_event`. The plot of the limit order books is depicted in Figure 1. There is significant crossed markets in both the CLOB and individual LOBs.

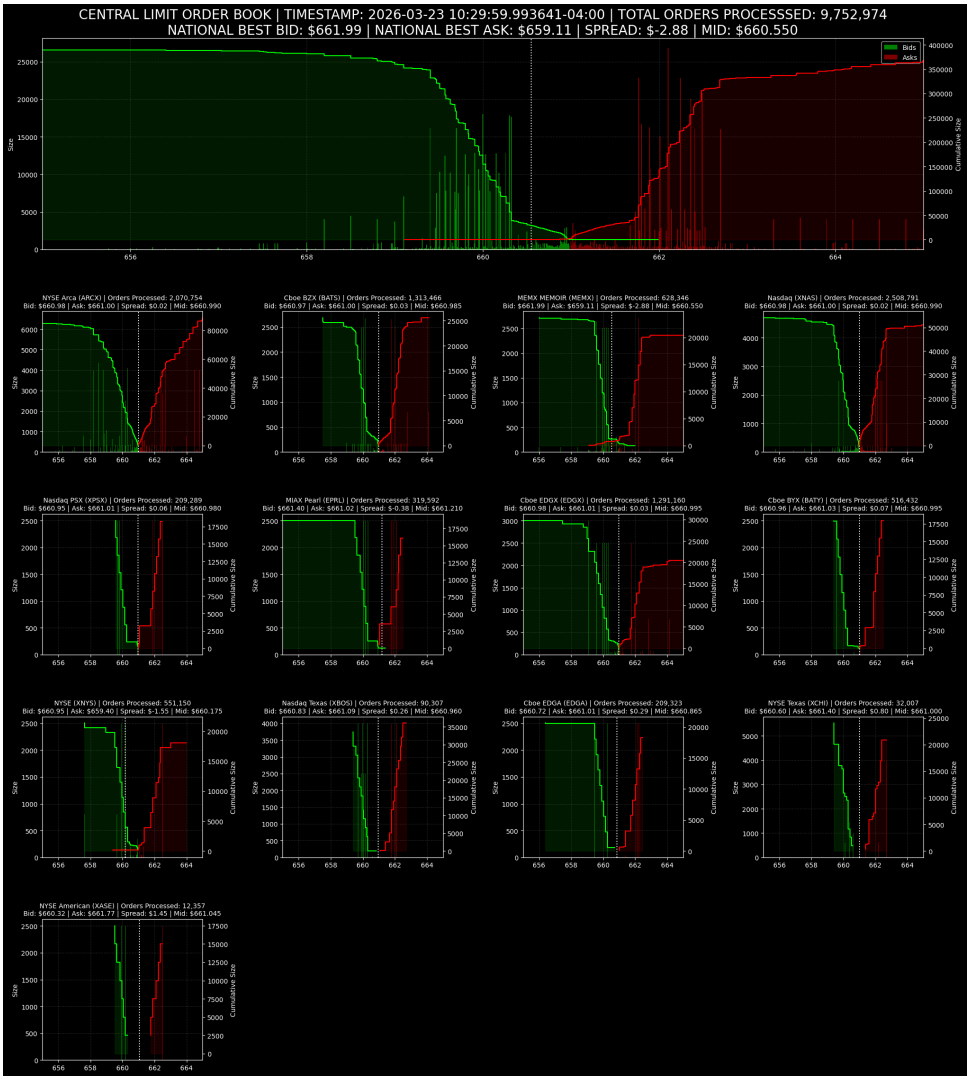

Figure 1: State of the Central Limit Order Book (CLOB) and 13 independent LOBs at the end of a 30-minute replay ordered by (`ts_event`).

Case 2: Reconstruction based on cumulative maximum timestamp (Jitter Corrected).
In the Jupyter Notebook `backtest_removed_jitter.ipynb`, we reconstruct the limit order book according to the order of `cumulative_maximum_timestamp_sorted_by_sequence` which is calculated using the methodology explained in Section 4. The result is that we do not get any crossed markets and the central limit order book represents an accurate portrayal of the real time period.

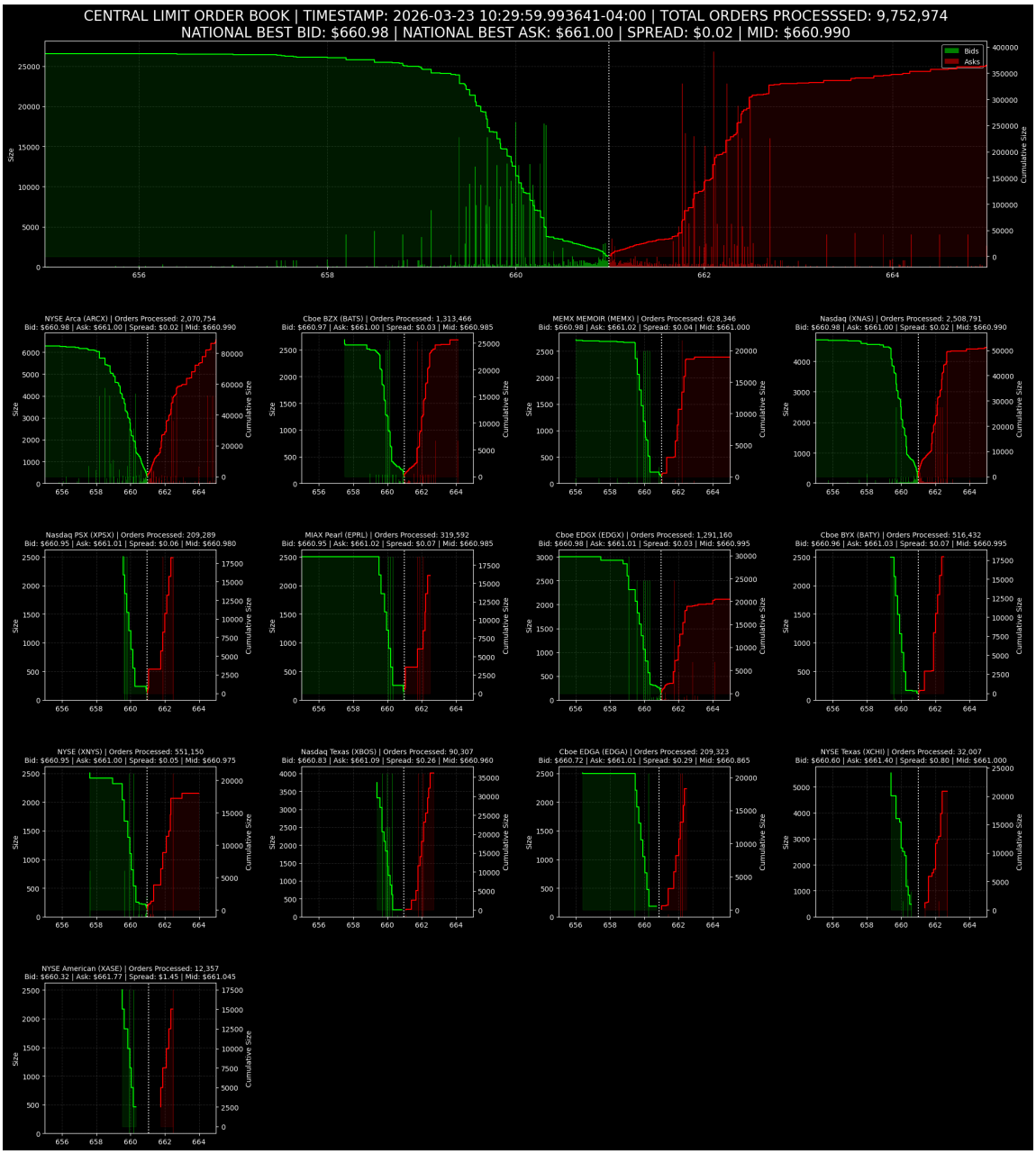

Figure 2: State of the Central Limit Order Book (CLOB) and 13 independent LOBs at the end of a 30-minute replay ordered by (`cumulative_maximum_timestamp_sorted_by_sequence`).

6 Conclusion

To construct accurate limit order books during backtesting, Market-By-Order data must first be sorted by the cumulative maximum timestamp.